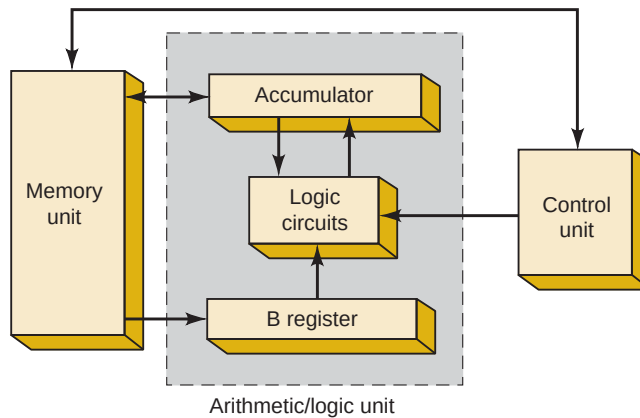


FIGURE 6-4 Functional parts of an ALU.



data that are stored in the memory and to execute arithmetic and logic operations on these data according to instructions from the control unit.

The arithmetic/logic unit contains at least two flip-flop registers: the *B register* and the **accumulator register**. It also contains combinational logic, which performs the arithmetic and logic operations on the binary numbers that are stored in the *B register* and the accumulator. A typical sequence of operations may occur as follows:

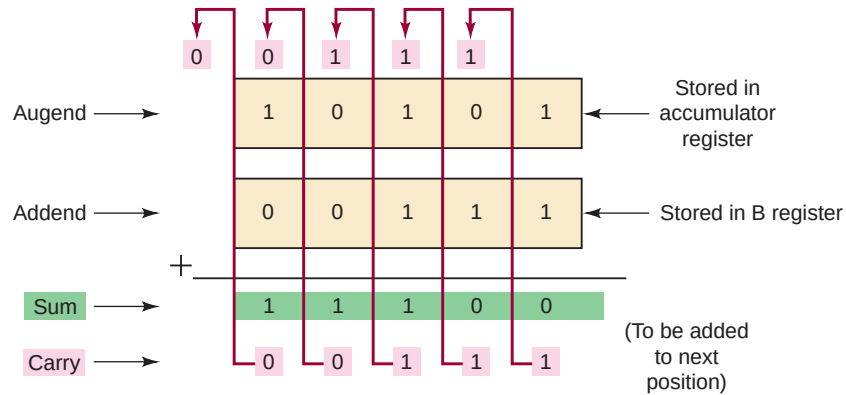
1. The control unit receives an instruction (from the memory unit) specifying that a number stored in a particular memory location (address) is to be added to the number presently stored in the accumulator register.
2. The number to be added is transferred from memory to the *B register*.
3. The number in the *B register* and the number in the accumulator register are added together in the logic circuits (upon command from the control unit). The resulting sum is then sent to the accumulator to be stored.
4. The new number in the accumulator can remain there so that another number can be added to it or, if the particular arithmetic process is finished, it can be transferred to memory for storage.

These steps should make it apparent how the accumulator register derives its name. This register “accumulates” the sums that occur when performing successive additions between new numbers acquired from memory and the previously accumulated sum. In fact, for any arithmetic problem containing several steps, the accumulator usually contains the results of the intermediate steps as they are completed as well as the final result when the problem is finished.

6-10 PARALLEL BINARY ADDER

Computers and calculators perform the addition operation on two binary numbers at a time, where each binary number can have several binary digits. Figure 6-5 illustrates the addition of two five-bit numbers. The **augend** is stored in the accumulator register; that is, the accumulator contains five FFs, storing the values 10101 in successive FFs. Similarly, the **addend**, the number that is to be added to the augend, is stored in the *B register* (in this case, 00111).

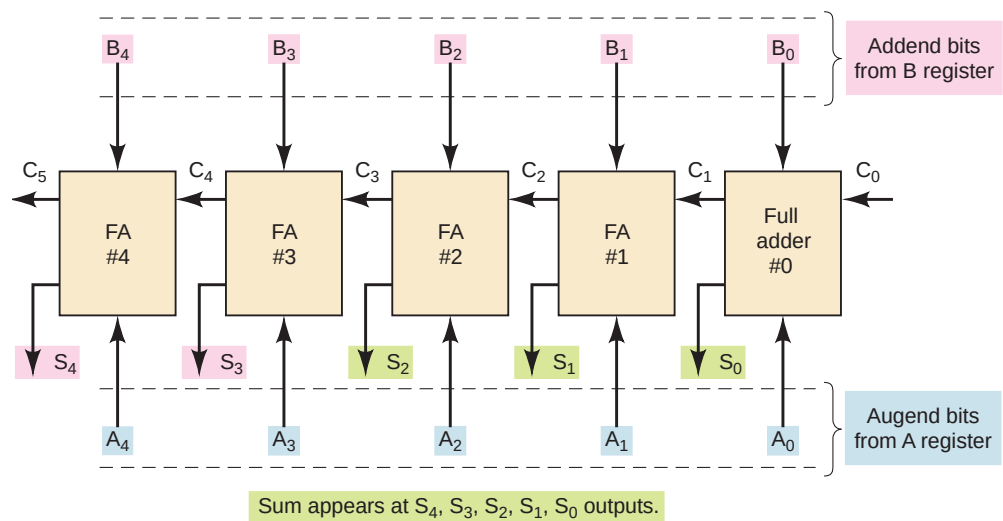
The addition process starts by adding the least significant bits (LSBs) of the augend and addend. Thus, $1 + 1 = 10$, which means that the *sum* for that position is 0, with a *carry* of 1.

FIGURE 6-5 Typical binary addition process.

This carry must be added to the next position along with the augend and addend bits in that position. Thus, in the second position, $1 + 0 + 1 = 10$, which is again a sum of 0 and a carry of 1. This carry is added to the next position together with the augend and addend bits in that position, and so on, for the remaining positions, as shown in Figure 6-5.

At each step in this addition process, we are performing the addition of three bits: the augend bit, the addend bit, and a carry bit from the previous position. The result of the addition of these three bits produces two bits: a *sum* bit, and a *carry* bit that is to be added to the next position. It should be clear that the same process is followed for each bit position. Thus, if we can design a logic circuit that can duplicate this process, then all we have to do is to use the identical circuit for each of the bit positions. This is illustrated in Figure 6-6.

In this diagram, variables A_4, A_3, A_2, A_1 , and A_0 represent the bits of the augend that are stored in the accumulator (which is also called the *A* register). Variables B_4, B_3, B_2, B_1 , and B_0 represent the bits of the addend stored in the *B* register. Variables C_4, C_3, C_2, C_1 , and C_0 represent the carry bits into the corresponding positions. Variables S_4, S_3, S_2, S_1, S_0 are the sum output bits for each position. Corresponding bits of the augend and addend are fed to a logic circuit called a **full adder (FA)**, along with a carry bit from the previous position. For example, bits A_1 and B_1 are fed into full adder 1 along

**FIGURE 6-6** Block diagram of a five-bit parallel adder circuit using full adders.

with C_1 , which is the carry bit produced by the addition of the A_0 and B_0 bits. Bits A_0 and B_0 are fed into full adder 0 along with C_0 . A_0 and B_0 are the LSBs of the augend and addend, so it appears that C_0 would always have to be 0 because there can be no carry into that position. We shall see, however, that there will be situations when C_0 can also be 1.

The full-adder circuit used in each position has three inputs: an A bit, a B bit, and a C bit. It also produces two outputs: a sum bit and a carry bit. For example, full adder 0 has inputs A_0 , B_0 , and C_0 , and it produces outputs S_0 and C_1 . Full adder 1 had A_1 , B_1 , and C_1 as inputs and S_1 and C_2 as outputs, and so on. This arrangement is repeated for as many positions as there are in the augend and addend. Although this illustration is for five-bit numbers, in modern computers the numbers usually range from 8 to 64 bits.

The arrangement in Figure 6-6 is called a **parallel adder** because all of the bits of the augend and addend are present and are fed into the adder circuits *simultaneously*. This means that the additions in each position are taking place at the same time. This is different from how we add on paper, taking each position one at a time starting with the LSB. Clearly, parallel addition is extremely fast. More will be said about this later.

REVIEW QUESTIONS

1. How many inputs does a full adder have? How many outputs?

2. Assume the following input levels in Figure 6-6: $A_4A_3A_2A_1A_0 = 01001$; $B_4B_3B_2B_1B_0 = 00111$; $C_0 = 0$.

(a) What are the logic levels at the outputs of FA #2?

(b) What is the logic level at the C_5 output?

6-11 DESIGN OF A FULL ADDER

Now that we know the function of the full adder, we can design a logic circuit that will perform this function. First, we must construct a truth table showing the various input and output values for all possible cases. Figure 6-7 shows the truth table having three inputs, A , B , and C_{IN} , and two outputs, S and C_{OUT} . There are eight possible cases for the three inputs, and for each

FIGURE 6-7 Truth table for a full-adder circuit.

